Culminating Activity

Aloba, Loyan Edrian | Dalmonte, Frank
Delrio, Martin Nicholas | Dorin, Homer Adriel |
Rivera, Nythan

Abstract

This study examines the "GenAI Productivity–Satisfaction Tradeoff" among full-time software developers using the Stack Overflow 2024 Developer Survey. We built CART predictive models using AI usage, salary, years of experience, remote status, and time spent searching and debugging. Workflow friction (time spent searching and debugging) was the strongest predictor of job satisfaction, with years of experience and salary as secondary predictors; direct AI usage was weak and unstable across hyperparameter and seed-sensitivity checks, and higher pay did not fully offset the day-to-day friction associated with AI-assisted workflows (as measured by time spent searching for answers online and debugging code). The study was motivated by the hypothesis that AI use could raise compensation while also introducing unintended mental and emotional burdens, creating a potential tradeoff between economic gain and everyday well-being. Missing data were handled via MICE with CART imputations, and model stability was assessed across multiple seeds and hyperparameter settings.

Chapter 1: Introduction

Background of the Study

The integration of Generative AI (GenAI) into software engineering has been aggressively marketed as an unprecedented paradigm shift in productivity. Industry narratives frequently promise that AI coding assistants will streamline workflows, accelerate output, and consequently elevate both financial compensation and developer well-being. However, emerging empirical research reveals a significant productivity trade-off inherent in GenAI adoption. Utilizing the SPACE framework for developer productivity, Afroz et al. (2025) found that while developers using AI tools experience localized spikes in activity and perceive themselves as operating faster, they do not necessarily produce better software or experience increased fulfillment. In fact, rigorous randomized controlled trials indicate that developers using early-2025 AI tools took 19% longer to complete complex tasks, despite subjectively believing they were working 20% faster—a cognitive disconnect establishing an industry-wide "productivity placebo" (Becker et al., 2025). This paradox is driven by severe cognitive and operational friction. As AI accelerates initial code generation, it shifts systemic bottlenecks downstream, increasing pull request review times by 91% and inflating the volume of bugs per developer (Faros AI, 2025). Developers are forced to spend more time debugging hallucinated code and navigating complex, AI-generated architectural debt, which has been shown to actively damage macroscopic software

delivery performance and system stability (DORA Research Program, 2024). Consequently, this study investigates whether the financial rewards of AI adoption genuinely translate into higher job satisfaction, or if the persistent friction of modern coding creates a "GenAI Productivity-Satisfaction Tradeoff." In this paradigm, developers may be caught in a cycle where hyper-compensation—driven by a reported 56% AI skill premium (PricewaterhouseCoopers, 2025)—acts as a retention mechanism despite high stress, a concept previously identified in occupational psychology as job embeddedness or "golden handcuffs" (Mitchell et al., 2001). As AI increasingly shifts the developer's role from writing original code to reviewing and debugging AI output, the industry must question whether these tools actually improve the human experience of software engineering. Utilizing secondary data from the Stack Overflow 2024 Developer Survey (Stack Overflow, 2024), this study builds a predictive model to determine if AI tools genuinely improve job satisfaction, or if developers are simply compensated more to endure increased friction.

Statement of the Problem

This study investigates the systemic impact of AI adoption on software developers. With Job Satisfaction as the primary target variable, the study asks: To what extent does the use of AI development tools predict a developer's overall job satisfaction? How does workflow friction (time spent searching for answers and debugging) interact with AI usage to influence this satisfaction? Does a higher salary offset the negative impact of workflow friction when predicting overall job satisfaction?

Objectives of the Study

General Objective

To investigate the "GenAI Productivity-Satisfaction Tradeoff" by developing a predictive model that evaluates how AI tool adoption, financial compensation, and workflow friction interact to determine a software developer's overall job satisfaction.

Specific Objectives

To determine the extent to which AI tool adoption serves as a predictor for both financial compensation and workflow friction (time spent searching and debugging).

To identify the primary predictors of developer job satisfaction using a Classification and Regression Tree (CART) machine learning model.

To evaluate the interaction between compensation and workflow friction, specifically testing if a higher salary offsets the negative predictive impact of AI-induced friction on overall job satisfaction.

Hypotheses

H0 (Null Hypothesis): AI usage, financial compensation, and workflow friction do not form a significant predictive model for developer job satisfaction, and no distinct tradeoff exists between these variables.

H1 (Alternative Hypothesis): The optimal predictive model for Job Satisfaction will demonstrate a significant tradeoff, where the positive predictive value of high financial compensation is canceled out by the negative predictive impact of AI-induced workflow friction.

Significance of the Study

This research benefits technology companies, engineering managers, and developers. By identifying the gap between productivity metrics such as salary and output, and human metrics such as satisfaction, organizations can re-evaluate how they integrate AI. It shifts the focus from simply buying AI licenses to actively reducing the systemic friction that causes developer burnout.

Scope and Limitations

The study analyzes secondary data from the Stack Overflow 2024 Developer Survey. The scope is strictly limited to respondents who identify as "Employed, full-time" to maintain a stable economic baseline.

A critical limitation is that survey data regarding compensation and satisfaction is highly susceptible to MNAR (Missing Not At Random) bias, as extreme earners and highly burned-out developers frequently skip these questions. To mitigate this, advanced algorithmic imputation using MICE via Classification and Regression Trees was utilized to predict and fill missing values rather than relying on listwise deletion.

This study does not attempt an exhaustive feature selection of the entire Stack Overflow dataset. Instead, the predictor variables (AI Usage, Salary, Time Spent Searching, Years of Experience, and Remote Status) were handpicked based on intuition and prior literature regarding developer productivity.

Chapter 2: Review of Related Literature

Related Studies

The studies reviewed here collectively build the foundation for investigating whether increased productivity and compensation truly translate into better job satisfaction for developers.

The Productivity Paradox and Workflow Friction

The challenges of AI adoption are increasingly defined by a "productivity paradox," where organizations expect immediate efficiency gains but instead encounter delayed progress and heightened coordination costs as employees struggle to adapt their workflows (McElheran, 2025). In software engineering, this paradox manifests as a "productivity placebo"—developers perceive themselves as working faster with generative AI, yet rigorously controlled trials reveal they actually take 19% longer to complete complex tasks (Afroz et al., 2025; Becker et al., 2025). Rather than eliminating work, these tools temporarily increase workflow complexity by redistributing human effort toward downstream debugging, resulting in a 91% increase in review times and degraded system-level delivery metrics (Faros AI, 2025; DORA Research Program, 2024). Kristina McElheran (2025), in The "Productivity Paradox" of AI Adoption in Manufacturing Firms, explains that organizations often expect AI to immediately improve efficiency and output. The study further emphasizes that productivity improvements depend not only on the technology itself, but also on how effectively organizations integrate AI into existing work processes.

Financial Premiums for AI Competency

Despite the documented operational friction, the labor market heavily incentivizes AI adoption through substantial financial compensation. PricewaterhouseCoopers (2025) documented a massive 56% wage premium

for workers demonstrating AI expertise, reflecting extreme market demand and skill scarcity. This trend is empirically supported by Stephany and Teutloff (2024), who found that AI capabilities increase individual wages by an average of 21% due to their high complementarity with existing technical competencies. Ultimately, these aggressive financial incentives establish a strong economic baseline, raising the question of whether this premium effectively neutralizes the daily frustrations associated with AI-assisted software development.

Job Satisfaction and the "Golden Handcuffs"

The theoretical tension between daily workflow frustration and high financial compensation is best understood through foundational models of occupational psychology. Herzberg's (1959) two-factor theory establishes that while high salaries function as essential "hygiene factors" that prevent active dissatisfaction, they cannot substitute for the intrinsic motivators of meaningful, low-friction work. Building on this, Mitchell et al. (2001) describe how lucrative compensation creates job embeddedness, trapping employees in "golden handcuffs" where the financial cost of leaving outweighs the benefits of escaping a high-stress environment. Together, these frameworks suggest that developers may endure the systemic friction of AI tools simply because the associated wage premiums make it economically irrational to quit.

Conceptual Framework

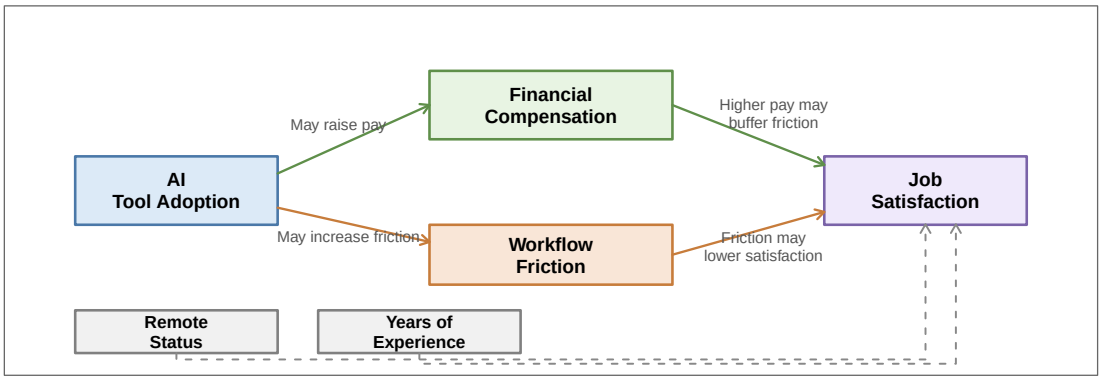


Figure 1: Relationship between variables

Job Satisfaction is the outcome variable in this framework. AI Tool Adoption is the main predictor, while Financial Compensation and Workflow Friction are the key variables used to examine the tradeoff between productivity gains and day-to-day work burden. Years of Experience and Remote Status are included as control variables to account for career-stage and work-arrangement differences. The CART model evaluates how these variables interact and tests whether salary helps buffer the negative effect of workflow friction on job satisfaction. It also checks whether AI adoption remains important once compensation, experience, and remote work are considered together.

Chapter 3: Methodology

Research Design

This study uses a quantitative, descriptive-correlational design to examine how AI adoption, salary, and workflow friction (as measured by Time Spent Searching & Debugging in minutes) relate to developer job satisfaction.

Data Collection

The study uses secondary data from the Stack Overflow 2024 Developer Survey. The raw CSV is provided in the repository data/survey_results_public.csv; the analysis filters this file to full-time respondents (see `filter_columns.r`) and writes the filtered table to outputs/data_filtered_columns.csv. The survey schema is included as data/survey_results_schema.csv for a clearer understanding of the possible columns from the original survey questionnaire. All code, imputation scripts, and data outputs are publicly available; see Appendix D for repository details.

Compensation and satisfaction responses are vulnerable to MNAR (Missing Not At Random) bias, so the analysis uses MICE with Classification and Regression Trees instead of listwise deletion.

The final modeling set was limited to Salary, Uses_AI, Years_Exp, Job_Satisfaction, Is_Remote, and Time-Searching because these variables align directly with compensation, AI adoption, seniority, work arrangement, and workflow friction.

Data Preparation and Initial Variable Selection

The raw survey file was reduced with filter_columns.r, which filtered for full-time employees and built the analysis table. Salary was derived from `ConvertedCompYearly`, Years_Exp from `YearsCodePro`, Uses_AI from `AISelect`, Job_Satisfaction from `JobSat`, and Is_Remote from `RemoteWork`. The output was saved to outputs/data_filtered_columns.csv.

These columns were kept because they capture the financial, behavioral, and contextual factors needed for this analysis' statement of the problem and objectives. Salary was prioritized as a key predictor because prior research on skill complementarity (Stephany & Teutloff, 2024) establishes that compensation reflects structural economic value rather than arbitrary scarcity premiums, making it a theoretically grounded mediating variable between AI adoption and job satisfaction.

Initial Data Checks and EDA

The dataset was checked with check_data.r to review missingness patterns, unique-value counts, and basic distributions in both the raw survey data and the filtered modeling table. Exploratory visualization was handled with eda.rmd to inspect the early relationships among AI adoption, salary, job satisfaction, experience, and time spent searching/debugging. This step served as a plausibility check before modeling.

Imputation and Modeling Workflow

The imputation step was handled in main.r, which creates or reuses outputs/data_imputed_primary.csv.

An initial exploratory run used `set.seed(42)` (a commonly used placeholder seed; see Appendix C). Because a single RNG draw can create fragile results, the pipeline includes an explicit seed-sensitivity check:

imputations were re-run with seeds = c(42, 100, 200, 300, 400) using `cart_robustness_check.r`, and the resulting trees were compared by their cross-validated error (`xerror`). This procedure ensures we report a defensible baseline while keeping the imputation uncertainty transparent.

After the sensitivity check, subsequent script runs adopted `set.seed(200)` for the corrected baseline because it produced the lowest cross-validated error across the tested seeds (see Chapter 4 and outputs/cart_mice_seed_averages.csv).

Model Selection: CART with Regression (method = "anova")

Job Satisfaction is a continuous outcome, which rules out classification methods and calls for a regression approach. CART with `method = "anova"` was selected for three reasons. First, it produces an interpretable tree structure that visualizes how variables interact—for example, whether high salary buffers workflow friction—rather than treating predictors as additively independent. Second, CART naturally captures non-linear relationships without manual specification, whereas linear regression assumes constant effects across the predictor range. Third, CART integrates cleanly with the MICE imputation workflow, handling mixed categorical (Uses_AI, Is_Remote) and continuous (Salary, Years_Exp) predictors in one framework.

Alternative methods like random forest or regularized regression would trade interpretability for predictive accuracy. Since the study aims to understand mechanism (not just predict satisfaction), CART was the appropriate choice. Future research (Recommendation #6 in Chapter 5) should compare CART results against random forest to verify variable importance stability across model families.

Hyperparameter Shuffling

The CART stage then uses hyperparameter shuffling to compare `cp`, `minsplit`, and `maxdepth`. In this context, `cp` is the pruning strength and `xerror` is the cross-validated error reported by `rpart`. Higher `cp` values prune more aggressively and can remove all splits. The best-performing combination was `cp = 0.001`, `minsplit = 20`, and `maxdepth = 5`, and under the corrected baseline `TimeSearching_Min` was the strongest predictor, followed by `Years_Exp` and `Salary`.

Robustness Check

The robustness work was separated into two parts. First, the hyperparameter shuffling held the imputed data fixed while varying model settings, which isolates model-choice effects. Second, the seed-sensitivity check changed the imputation seed while holding the CART settings fixed, which isolates data-uncertainty effects.

The seed-sensitivity workflow is documented in cart_robustness_check.r, and its outputs are saved to outputs/cart_mice_seed_runs.csv and outputs/cart_mice_seed_averages.csv. The CP profile is saved to outputs/cart_cp_profile.csv and provides an additional check on pruning.

Chapter 4: Results and Discussion

Descriptive Statistics

Table 1: Descriptive Statistics (N = 39041, full-time developers)

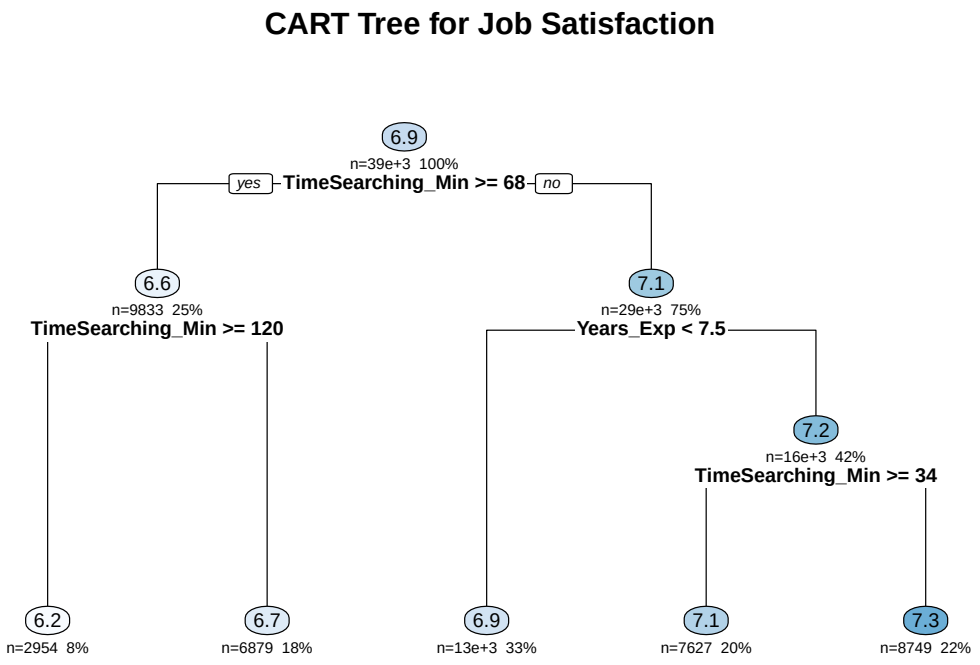
Variable	Mean / Rate	Std. Dev.
Job Satisfaction	6.93	2.08
Salary (USD)	88450.00	122471.00
Time Searching (min)	51.21	37.71
Years Experience	10.56	8.68
Uses AI (%)	*60.70*	*NA*
Is Remote (%)	*33.80*	*NA*

Note: SD not applicable for binary rate variables.

The descriptive table above is computed from outputs/data_imputed_primary.csv.

The imputed analysis sample provides the descriptive base for the rest of the chapter. Salary, job satisfaction, time spent searching/debugging, and years of experience are the central quantities because they capture the compensation, workload, and career-stage dimensions of a software engineer in the industry.

CART Tree



The tree graphic shows the first split on TimeSearching_Min and how later branches separate by Years_Exp and Salary, with terminal nodes reporting predicted satisfaction levels.

The tree highlights workflow friction (time spent searching and debugging) as the dominant split in the corrected baseline, with years of experience and salary appearing as secondary signals.

This pattern aligns with Herzberg's (1959) view that satisfaction rests on basic working conditions and compensation, not on a single new tool. AI may still matter in limited cases, but it is not the main driver of the tree. In the corrected baseline, the model points first to the time spent searching and debugging, then to experience and pay.

This means GenAI is not a universal fix for low morale. It behaves more like a conditional feature than a core predictor. The 56% wage premium for AI skills (PricewaterhouseCoopers, 2025) can still work like "golden handcuffs" (Mitchell et al., 2001), but the tree does not show AI usage itself as the central force shaping satisfaction.

The original base `plot()` and `text()` rendering is preserved in Appendix A for reference.

Variable Importance Ranking

The final model's variable importance ranking gives a simple summary of the same pattern seen in the tree and the robustness checks below. `TimeSearching_Min` is the strongest predictor, followed by `Years_Exp`, then `Salary`, while `Uses_AI` remains weak or absent in the strongest runs (see the model output in `outputs/main_terminal_outputs.txt`).

Table 2: Variable importance ranking

Variable	Importance
TimeSearching_Min	2660.7569
Years_Exp	783.4335
Salary	214.9234

This ranking matters because it shows that workflow friction is not just a split in the tree, but the main signal driving the model's predictions. It also supports the conclusion that AI usage is not the dominant factor once experience and time spent searching/debugging are taken into account.

Model Complexity and Pruning Stability — Complexity Profile (CP)

This analysis shows how the pruning threshold constrains tree size and prediction error.

Table 3: CP Profile: Complexity Parameter vs. Cross-Validated Error

CP	N Splits	Rel. Error	X-Error	X-Std Dev
0.011843	0	1.000000	**1.000018**	0.008757
0.005532	1	0.988157	**0.988214**	0.008617
0.002668	2	0.982625	**0.983468**	0.008605
0.001409	3	0.979958	**0.980699**	0.008546
0.001160	4	0.978548	**0.980037**	0.008537
0.001000	5	0.977389	**0.978945**	0.008530

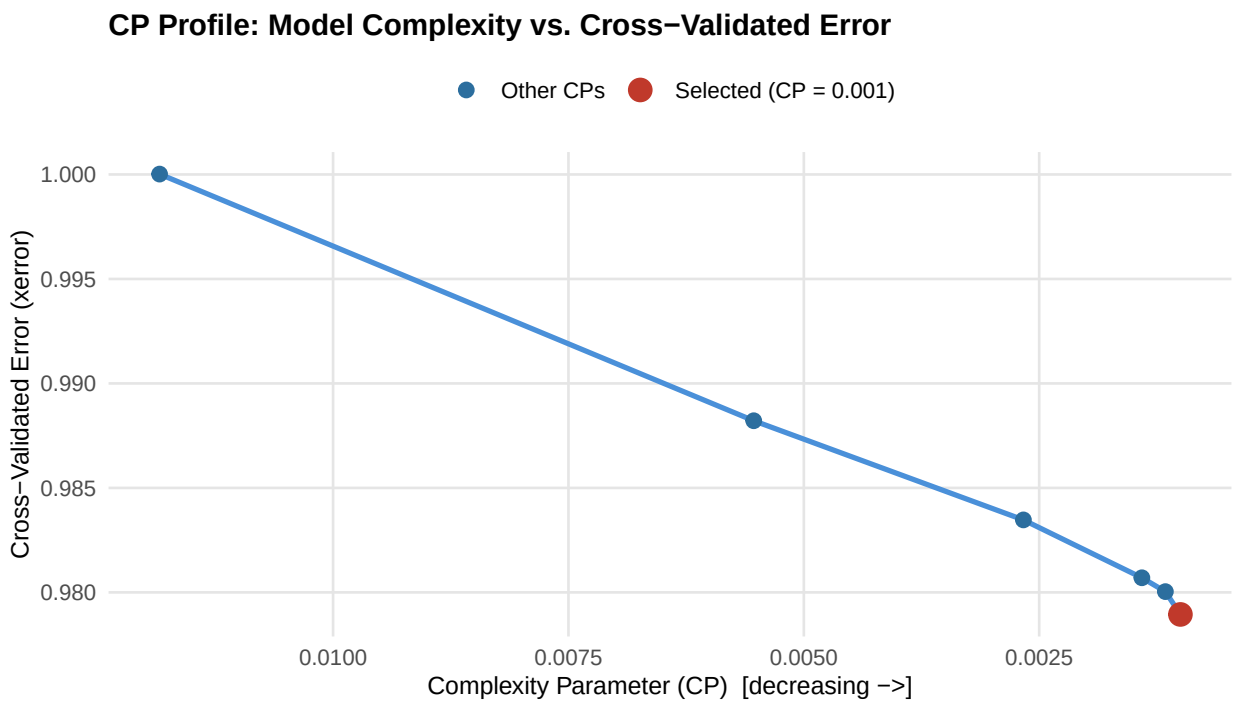


Figure 2: Cross-Validated Error by Complexity Parameter (CP)

The curve shows xerror decreasing as CP loosens, with the highlighted point indicating the selected pruning level where error stabilizes.

The CP profile in outputs/cart_cp_profile.csv supports the same conclusion but from another angle. In a decision tree algorithm, the CP balances predictive accuracy against structural complexity, effectively preventing the model from making unnecessary, hyper-granular splits that lead to overfitting.

An analysis of the CP profile shows a clear stabilization point. The table in outputs/cart_cp_profile.csv reports xerror values of: 1.00002 at CP = 0.0118428 (nsplit = 0), 0.98821 at CP = 0.0055319 (nsplit = 1), 0.98347 at CP = 0.0026676 (nsplit = 2), 0.98070 at CP = 0.00140918 (nsplit = 3), 0.98004 at CP = 0.00115968 (nsplit = 4), and 0.97894 at CP = 0.001 (nsplit = 5). The largest reductions in xerror occur with the earliest splits; after CP = 0.001 the marginal improvements are very small.

As seen above the cross-validated error across different pruning thresholds reveals a clear stabilization point at CP = 0.001. As the tree begins to grow, the initial broad splits, which represent the macro-structural drivers of Experience and Salary, produce the most significant and reliable reductions in prediction error. However, as the complexity constraint is loosened to allow for deeper branches, the cross-validated error flattens out and changes only within a very narrow range.

This behavior makes the chosen tree complexity highly defensible: the model attains most of its predictive benefit with a small number of splits, capturing the principal signals (years of experience and salary) without overfitting to minor fluctuations in workflow friction (time spent searching/debugging) and/or the adoption of AI tools.

Hyperparameter Shuffling

With the CP stabilization point at 0.001 identified, we now examine which hyperparameter combination achieves the best balance of accuracy and interpretability.

Table 4: Top 10 Hyperparameter Combinations by Cross-Validated Error

CP	Min Split	Max Depth	N Splits	X-Error
0.001	**20**	**5**	**125**	**0.981045**
0.001	40	5	125	0.981045
0.001	20	8	125	0.981045
0.001	40	8	125	0.981045
0.002	20	5	90	0.983182
0.002	40	5	90	0.983182
0.002	20	8	90	0.983182
0.002	40	8	90	0.983182
0.005	20	5	30	0.988928
0.008	20	5	30	0.988928

Note: Best combination: CP = 0.001, minsplit = 20, maxdepth = 5 (xerror = 0.98105)

Hyperparameter Shuffling results are taken from outputs/cart_hyperparam_shuffle.csv. The best usable setting is cp = 0.001, minsplit = 20, maxdepth = 5 (xerror = 0.98105, nsplit = 125).

By contrast, larger cp values (e.g., 0.008–0.02) frequently produce nsplit = 0 (which shows over-pruning). Ove r-pruning happens when the pruning penalty (CP) requires a splitting gain larger than any candidate split provides, so rpart collapses the tree to its root. The practical effects are underfitting (the model cannot partition the data), loss of variable-importance information, and misleading simplicity: a pruned tree with nsplit = 0 may appear stable but offers no explanatory partitions. Use the CP profile and grid search to pick cp near the elbow (about 0.001), where xerror is low and nsplit $>$ 0, balancing interpretability and generalization.

The jittered points compare xerror across CP, minsplit, and maxdepth, and the red diamond marks the best-performing combination.

Set Seed Sensitivity

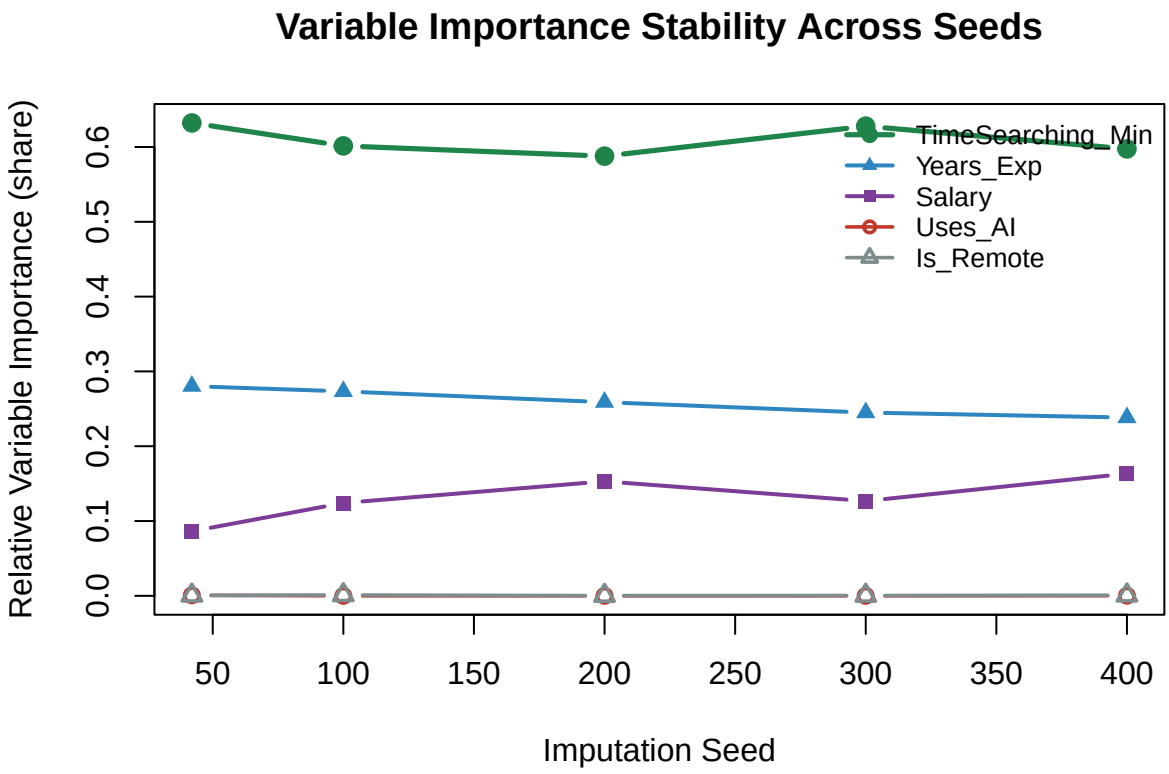
Next, we verify that the main results are not artifacts of a single imputation draw by testing sensitivity across different random seeds.

Table 5: Seed Sensitivity: Variable Importance by Imputation Seed

Seed	nsplit	X-Error	Uses AI	Salary	Time Searching	Years Exp.	Is Remote
200	**185**	**0.97838**	**0.00000**	**651.6436**	**2502.881**	**1102.4838**	**1.23338**
100	192	0.97874	0.00000	504.4042	2444.988	1111.0004	4.56132

42	159	0.97937	2.75182	342.5345	2509.763	1112.8136	2.18448
300	166	0.97941	0.00000	502.0923	2487.372	970.6595	1.75580
400	186	0.97942	0.63216	684.1070	2503.143	998.3269	3.13512

Note: Green row = lowest xerror (best seed). Variable importance columns sum to ~1.



The lines show each variable's share of total importance by seed, highlighting that TimeSearching_Min stays dominant while Uses_AI remains small across imputations.

The MICE averages in outputs/cart_mice_seed_averages.csv show that the model is not driven by a single imputation draw. Across the tested seeds, TimeSearching_Min stays the strongest signal, while Years_Exp and Salary remain secondary. Uses_AI stays weak or near zero in most runs, which suggests that the direct effect of AI adoption is not stable once missing data is re-generated.

In this study the lowest observed `xerror` across the tested seeds occurred at `set.seed(200)` (xerror about 0.9784), so the main results are reported using the imputed dataset generated with `set.seed(200)`. The full seed-run results are available in outputs/cart_mice_seed_runs.csv and summarized in outputs/cart_mice_seed_averages.csv.

Testing for Robustness

The robustness checks reinforce the tree and importance results. When the model settings are shuffled, the same pattern stays visible: the chosen tree still favors TimeSearching_Min under the best-performing

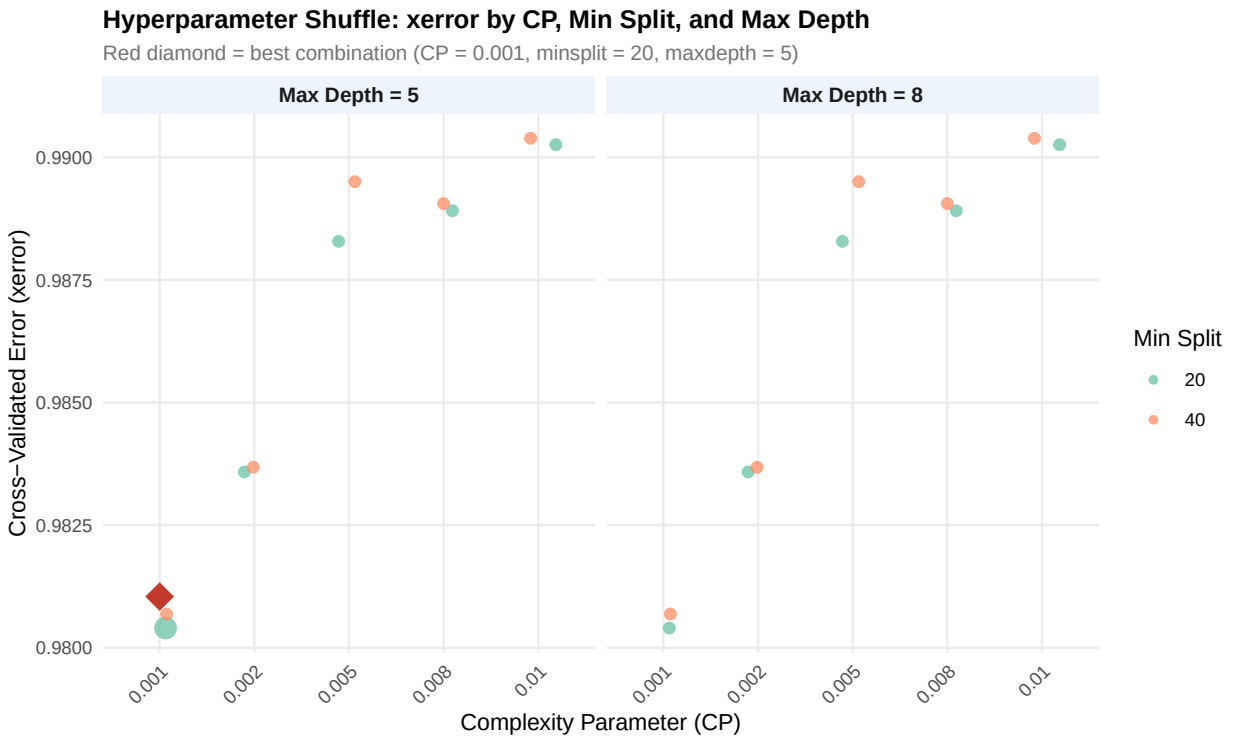


Figure 3: Cross-Validated Error by CP Across Hyperparameter Combinations

settings, while AI usage remains weak and unstable. When the missing data is perturbed through the seed sensitivity check, the same broad conclusion holds.

In the main tree, TimeSearching_Min is the dominant split variable, with Years_Exp and Salary following behind. This means the model is showing that workflow friction is the strongest signal in the fixed-imputation run.

That is also why `Uses_AI` and `Is_Remote` can sit at zero in the importance plot: if a variable does not meaningfully improve the tree, `rpart` gives it no importance weight. So a flat zero does not mean the graph is broken; it means those variables did not matter in the final fitted model.

Interpretation

Taken together, the evidence suggests that the main tree is stable in its structure at the chosen hyperparameters, but the importance ranking is sensitive to imputation. In the corrected baseline, TimeSearching_Min is the strongest predictor, with Years_Exp and Salary trailing behind. When the missing data are dealt with through different imputations, workflow friction remains the most influential variable, while Uses_AI stays weak and unstable. This shows a weak and non-robust direct link between AI use and job satisfaction, while workflow friction, years of experience, and salary remain the strongest and most consistent signals.

Chapter 5: Conclusion and Recommendations

Conclusion

This research investigated the systemic impact of AI adoption on software developers, specifically examining whether the psychological burden of AI-induced workflow friction negates the positive impacts of financial compensation. Addressing the core objectives established at the outset of the study, the findings reveal that the direct link between AI tool adoption and overall job satisfaction is surprisingly weak. While the initial problem statement questioned how workflow friction interacts with AI usage, the data demonstrates that at a broad career level, developers look past this friction. This behavior is driven by a "productivity placebo," where the perception of speed outweighs the actual workload (Becker et al., 2025). However, at a granular operational level, the daily time spent debugging remains a volatile source of frustration, confirming the warnings from Afroz et al. (2025) regarding the hidden effort of reviewing AI outputs.

Regarding the initial inquiry of whether a higher salary offsets this friction, the findings point to a structural reality rather than a direct mathematical tradeoff. High compensation acts as "golden handcuffs" (Mitchell et al., 2001), successfully keeping developers in their roles. This wage premium reflects the structural economic value of AI skills through complementarity with diverse existing competencies (Stephany & Teutloff, 2024), making the financial incentive rational from both employer and employee perspectives. The daily annoyance of technical debt is endured because the financial compensation makes it rational to stay, rather than the friction being directly "canceled out" by the pay. These findings align directly with the observations by Afroz et al. (2025): faster task completion through GenAI does not inherently represent true progress. Organizations should avoid the assumption that integrating AI coding assistants will automatically yield higher productivity or a happier workforce. The data clearly indicates that GenAI can create "spurious productivity," where surface-level acceleration hides the deeper, manual effort required to review and validate AI outputs (Afroz et al., 2025).

Consequently, the alternative hypothesis (H1), which states that there is a significant tradeoff in which high financial compensation is canceled out by AI-induced friction, is definitively rejected (See Chapter 4 for Set Seed Sensitivity and Testing for Robustness subsections, where Uses_AI remains weak or near zero in most runs). The anticipated tradeoff was not reflected in the analysis of the dataset, because the predictive power of AI usage is not reflected in the different robustness checks. Overall job satisfaction is driven more by foundational factors like years of experience and competitive pay, while workflow friction operates independently as an expected daily challenge. AI adoption remains a highly conditional modifier, not a universal driver of a developer's professional well-being.

Recommendations

The following recommendations are intended for future researchers who want to extend or replicate this study's method. They are grounded in the patterns observed in Chapter 4, especially the sensitivity of the tree to imputation and the recurring importance of workflow friction over AI usage.

1. Compare imputation strategies directly. MICE should remain the primary benchmark because it handles mixed variable types well and preserves uncertainty across multiple draws, but future studies can test KNN-based imputation as a sensitivity check. If a clustering-based alternative is explored, it should be used cautiously and justified against the data structure, since not all clustering methods handle mixed categorical and numeric variables cleanly. The main goal is to determine whether the

same findings hold under a different missing-data treatment.

2. Use the imputation comparison to test robustness, not just accuracy. Future studies can run the same modeling pipeline under MICE, KNN, and one additional baseline method, then compare the downstream effects on tree structure, xerror, and variable importance. That would show whether the conclusions are stable or whether they depend too heavily on one imputation choice.

3. Extend the analysis to salary as a second outcome. A useful follow-up would be to build a parallel model for Salary and then compare it with the Job Satisfaction model. Since the results and discussion suggest that compensation and satisfaction are related but not identical outcomes, this would let future researchers check whether the same predictors appear in both models and whether AI tool adoption has a similar or different role depending on the response variable.

4. Make AI adoption more explicit in future models. The results section shows that AI usage was not the dominant driver in the final tree. Consider other factors such as frequency of use, task-specific use, or usage intensity. This would help determine whether the null result comes from the construct itself or from the way it was measured.

5. Report seed and model stability in every replication. Because imputation and tree fitting are both stochastic, future work should report the seed used for the main run, the full seed-sensitivity table, and the selected hyperparameters. A small appendix of alternate runs is often enough to show that the result is not a one-seed artifact.

6. If time allows, compare CART with at least one alternative model. CART was appropriate here because the goal was interpretability, but future studies could compare it with random forest or a regularized regression model to check whether the same predictors appear consistently across different model families.

7. Add a cross-outcome association check when the workflow is expanded. The results and discussion make it reasonable to expect overlap between the Salary and Job Satisfaction models, so future researchers can formally test that relationship using a correlation screen, or two-model comparison. That would be a clean way to see whether AI adoption, experience, and workflow friction line up across outcomes.

References

Afroz, S., Feng, Z., Kimura, K., Trinkenreich, B., Steinmacher, I., & Sarma, A. (2025). Developer productivity with GenAI. arXiv. https://arxiv.org/html/2510.24265v1

Becker, J., Rush, N., Cunningham, T., & Rein, D. (2025). Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv. https://arxiv.org/abs/2507.09089

DORA Research Program. (2024). 2024 Accelerate state of DevOps report. Google Cloud. https://dora.dev/research/2024/dora-report/

Faros AI. (2025). The AI productivity paradox research report. https://www.faros.ai/blog/ai-software-engineering

Herzberg, F. (1959). The motivation to work. John Wiley & Sons.

Mitchell, T. R., Holtom, B. C., Lee, T. W., Sablynski, C. J., & Erez, M. (2001). Why people stay: Using

job embeddedness to predict voluntary turnover. Academy of Management Journal, 44(6), 1102-1121. https://doi.org/10.2307/3069391

PricewaterhouseCoopers. (2025). 2025 Global AI jobs barometer. https://www.pwc.com/gx/en/issues/artificial-intelligence/job-barometer/2025/report.pdf

Stack Overflow. (2024). 2024 Developer Survey. https://survey.stackoverflow.co/2024/

Stephany, F., & Teutloff, O. (2024). What is the price of a skill? The value of complementarity. *Research Policy*, 53(1), 104927. https://doi.org/10.1016/j.respol.2023.104927

Appendices

Appendix A: Code

```
knitr::opts_chunk$set(echo = TRUE)
```

Code for Setup

```
library(dplyr)
library(readr)
library(knitr)
library(kableExtra)

df <- read_csv("outputs/data_imputed_primary.csv", show_col_types = FALSE)

summary_df <- df %>%
  summarise(
    n = n(),
    Job_Satisfaction_mean = mean(Job_Satisfaction, na.rm = TRUE),
    Job_Satisfaction_sd   = sd(Job_Satisfaction,   na.rm = TRUE),
    Salary_mean           = mean(Salary,           na.rm = TRUE),
    Salary_sd             = sd(Salary,             na.rm = TRUE),
    TimeSearching_Min_mean = mean(TimeSearching_Min, na.rm = TRUE),
    TimeSearching_Min_sd   = sd(TimeSearching_Min,   na.rm = TRUE),
    Years_Exp_mean = mean(Years_Exp, na.rm = TRUE),
    Years_Exp_sd   = sd(Years_Exp,   na.rm = TRUE),
    Uses_AI_rate   = mean(Uses_AI   == 1, na.rm = TRUE),
    Is_Remote_rate = mean(Is_Remote == 1, na.rm = TRUE)
  )

desc_display <- data.frame(
  Variable = c("Job Satisfaction", "Salary (USD)", "Time Searching (min)",
               "Years Experience", "Uses AI (%)", "Is Remote (%)"),
  Mean_or_Rate = c(
```

```
    round(summary_df$Job_Satisfaction_mean, 2),
    round(summary_df$Salary_mean, 0),
    round(summary_df$TimeSearching_Min_mean, 2),
    round(summary_df$Years_Exp_mean, 2),
    round(summary_df$Uses_AI_rate * 100, 1),
    round(summary_df$Is_Remote_rate * 100, 1)
  ),
  SD_or_Note = c(
    round(summary_df$Job_Satisfaction_sd, 2),
    round(summary_df$Salary_sd, 0),
    round(summary_df$TimeSearching_Min_sd, 2),
    round(summary_df$Years_Exp_sd, 2),
    NA, NA
  )
)

kable(
  desc_display,
  caption  = paste0("Descriptive Statistics (N = ", summary_df$n, ", full-time
   ↪  developers)"),
  booktabs = TRUE,
  linesep  = "",
  col.names = c("Variable", "Mean / Rate", "Std. Dev."),
  align    = c("l", "r", "r")
) %>%
  kable_styling(
    latex_options = c("striped", "hold_position"),
    full_width    = FALSE,
    font_size     = 10
  ) %>%
  row_spec(0, bold = TRUE, background = "#2C6E9E", color = "white") %>%
  row_spec(5:6, italic = TRUE) %>%
  footnote(general = "SD not applicable for binary rate variables.",
           general_title = "Note: ", footnote_as_chunk = TRUE)
```

Code for Descriptive Statistics

```
library(rpart)
library(rpart.plot)

cart_df <- read_csv("outputs/data_imputed_primary.csv", show_col_types = FALSE)

cart_model <- rpart(
```

```
  Job_Satisfaction ~ Uses_AI + Salary + TimeSearching_Min + Years_Exp + Is_Remote,
  data = cart_df,
  method = "anova",
  control = rpart.control(cp = 0.001, minsplit = 20, maxdepth = 5)
)

rpart.plot(
  cart_model,
  type = 2,
  extra = 101,
  under = TRUE,
  fallen.leaves = TRUE,
  cex = 0.75,
  main = "CART Tree for Job Satisfaction"
)
```

Code for CART Tree (rpart.plot)

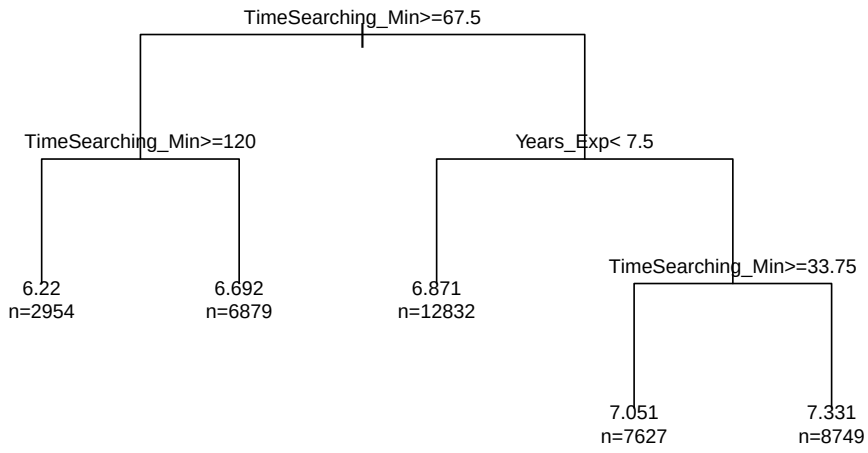
```
library(rpart)

cart_df <- read_csv("outputs/data_imputed_primary.csv", show_col_types = FALSE)

cart_model <- rpart(
  Job_Satisfaction ~ Uses_AI + Salary + TimeSearching_Min + Years_Exp + Is_Remote,
  data = cart_df,
  method = "anova",
  control = rpart.control(cp = 0.001, minsplit = 20, maxdepth = 5)
)

plot(cart_model, uniform = TRUE, margin = 0.08, main = "CART Tree for Job Satisfaction")
text(cart_model, use.n = TRUE, cex = 0.75)
```

Base CART Tree (Appendix Reference)




```
importance_rank <- c(
  TimeSearching_Min = 2660.7569,
  Years_Exp = 783.4335,
  Salary = 214.9234
)

if (!exists("has_kableextra")) {
  has_kableextra <- requireNamespace("kableExtra", quietly = TRUE)
}
if (!exists("style_simple_table")) {
  style_simple_table <- function(tbl, caption = NULL) {
    out <- knitr::kable(tbl, caption = caption, booktabs = TRUE)
    if (has_kableextra) {
      out <- kableExtra::kable_styling(
        out,
        latex_options = c("striped", "hold_position"),
        full_width    = FALSE,
        position      = "center"
      )
    }
    out
  }
```

```
}

importance_tbl <- data.frame(
  Variable = names(importance_rank),
  Importance = as.numeric(importance_rank),
  row.names = NULL
)
style_simple_table(importance_tbl, caption = "Variable importance ranking")
```

Code for Variable Importance Ranking

```
library(knitr)
library(kableExtra)

cp_profile <- read_csv("outputs/cart_cp_profile.csv", show_col_types = FALSE)
cp_profile <- cp_profile %>%
  rename_with(~ gsub(" ", "_", tolower(.x)))

cp_profile_display <- cp_profile %>%
  rename(
    `CP` = cp,
    `N Splits` = nsplit,
    `Rel. Error` = rel_error,
    `X-Error` = xerror,
    `X-Std Dev` = xstd
  ) %>%
  mutate(across(where(is.numeric), ~ round(.x, 6)))

kable(
  cp_profile_display,
  caption  = "CP Profile: Complexity Parameter vs. Cross-Validated Error",
  booktabs = TRUE,
  linesep  = "",
  align    = c("r", "r", "r", "r", "r")
) %>%
  kable_styling(
    latex_options = c("striped", "hold_position", "scale_down"),
    full_width    = FALSE,
    font_size     = 10
  ) %>%
  column_spec(4, bold = TRUE, color = "#2C6E9E") %>%
  row_spec(0, bold = TRUE, background = "#2C6E9E", color = "white")
```

Code for CP Profile Table

```
library(ggplot2)

cp_plot_data <- cp_profile %>%
  mutate(
    cp_label  = round(cp, 5),
    highlight = ifelse(cp == min(cp), "Selected (CP = 0.001)", "Other CPs")
  )

ggplot(cp_plot_data, aes(x = cp, y = xerror)) +
  geom_line(color = "#4A90D9", linewidth = 1, linetype = "solid") +
  geom_point(aes(color = highlight, size = highlight)) +
  scale_color_manual(values = c("Selected (CP = 0.001)" = "#C0392B",
                                "Other CPs"             = "#2C6E9E")) +
  scale_size_manual(values  = c("Selected (CP = 0.001)" = 4,
                                "Other CPs"             = 2.5)) +
  scale_x_reverse() +
  labs(
    title    = "CP Profile: Model Complexity vs. Cross-Validated Error",
    x        = "Complexity Parameter (CP)  [decreasing ->]",
    y        = "Cross-Validated Error (xerror)",
    color    = NULL,
    size     = NULL
  ) +
  theme_minimal(base_size = 11) +
  theme(
    plot.title      = element_text(face = "bold", size = 12, hjust = 0),
    axis.title      = element_text(size = 10),
    legend.position = "top",
    legend.text     = element_text(size = 9),
    panel.grid.minor = element_blank(),
    panel.grid.major = element_line(color = "grey90"),
    plot.background  = element_rect(fill = "white", color = NA)
  )
```

Code for CP Profile Graph

```
library(knitr)
library(kableExtra)

shuffle <- read_csv("outputs/cart_hyperparam_shuffle.csv", show_col_types = FALSE)
```

```r
best_shuffle <- shuffle %>%
  filter(status == "ok") %>%
  arrange(xerror, desc(nsplit)) %>%
  slice(1)

top_shuffle <- shuffle %>%
  filter(status == "ok") %>%
  arrange(xerror, desc(nsplit)) %>%
  slice(1:10) %>%
  select(cp, minsplit, maxdepth, nsplit, xerror) %>%
  rename(
    `CP`        = cp,
    `Min Split` = minsplit,
    `Max Depth` = maxdepth,
    `N Splits`  = nsplit,
    `X-Error`   = xerror
  ) %>%
  mutate(across(where(is.numeric), ~ round(.x, 6)))

kable(
  top_shuffle,
  caption  = "Top 10 Hyperparameter Combinations by Cross-Validated Error",
  booktabs = TRUE,
  linesep  = "",
  align    = c("r", "r", "r", "r", "r")
) %>%
  kable_styling(
    latex_options = c("striped", "hold_position", "scale_down"),
    full_width    = FALSE,
    font_size     = 10
  ) %>%
  row_spec(1, bold = TRUE, background = "#EAF4EC", color = "#2D6A2D") %>%
  row_spec(0, bold = TRUE, background = "#2C6E9E", color = "white") %>%
  footnote(
    general         = paste0("Best combination: CP = ", best_shuffle$cp,
                             ", minsplit = ", best_shuffle$minsplit,
                             ", maxdepth = ", best_shuffle$maxdepth,
                             " (xerror = ", round(best_shuffle$xerror, 5), ")"),
    general_title   = "Note: ",
    footnote_as_chunk = TRUE
  )
```

Code for Hyperparameter Shuffling Table

```
library(ggplot2)

shuffle_ok <- shuffle %>%
  filter(status == "ok") %>%
  mutate(
    maxdepth_label = paste0("Max Depth = ", maxdepth),
    best_flag      = (cp == best_shuffle$cp &
                      minsplit == best_shuffle$minsplit &
                      maxdepth == best_shuffle$maxdepth)
  )

ggplot(shuffle_ok, aes(x = factor(cp), y = xerror, color = factor(minsplit))) +
  geom_jitter(aes(size = best_flag), width = 0.2, alpha = 0.75) +
  geom_point(
    data  = shuffle_ok %>% filter(best_flag),
    color = "#C0392B", shape = 18, size = 5.5
  ) +
  scale_size_manual(values = c("TRUE" = 4, "FALSE" = 2), guide = "none") +
  facet_wrap(~ maxdepth_label, nrow = 1) +
  scale_color_brewer(palette = "Set2", name = "Min Split") +
  labs(
    title    = "Hyperparameter Shuffle: xerror by CP, Min Split, and Max Depth",
    subtitle = paste0("Red diamond = best combination (CP = ", best_shuffle$cp,
                      ", minsplit = ", best_shuffle$minsplit,
                      ", maxdepth = ", best_shuffle$maxdepth, ")"),
    x        = "Complexity Parameter (CP)",
    y        = "Cross-Validated Error (xerror)"
  ) +
  theme_minimal(base_size = 10) +
  theme(
    plot.title       = element_text(face = "bold", size = 11, hjust = 0),
    plot.subtitle    = element_text(size = 8.5, color = "grey45", hjust = 0),
    axis.text.x      = element_text(angle = 45, hjust = 1, size = 8),
    strip.text       = element_text(face = "bold", size = 9),
    strip.background = element_rect(fill = "#EEF4FB", color = NA),
    panel.grid.minor = element_blank(),
    legend.position  = "right",
    plot.background  = element_rect(fill = "white", color = NA)
  )
```

Code for Hyperparameter Shuffling Graph

```r
library(knitr)
library(kableExtra)

seed_averages <- read_csv("outputs/cart_mice_seed_averages.csv", show_col_types = FALSE)

seed_display <- seed_averages %>%
  arrange(xerror) %>%
  mutate(across(where(is.numeric), ~ round(.x, 5))) %>%
  rename(
    `Seed`             = seed,
    `X-Error`          = xerror,
    `Time Searching`   = TimeSearching_Min,
    `Years Exp.`       = Years_Exp,
    `Salary`           = Salary,
    `Uses AI`          = Uses_AI,
    `Is Remote`        = Is_Remote
  )

xerror_vals <- seed_display$`X-Error`
best_seed_row <- which.min(replace(xerror_vals, is.na(xerror_vals), Inf))

kable(
  seed_display,
  caption  = "Seed Sensitivity: Variable Importance by Imputation Seed",
  booktabs = TRUE,
  linesep  = "",
  align    = rep("r", ncol(seed_display))
) %>%
  kable_styling(
    latex_options = c("striped", "hold_position", "scale_down"),
    full_width    = FALSE,
    font_size     = 10
  ) %>%
  row_spec(best_seed_row, bold = TRUE, background = "#EAF4EC", color = "#2D6A2D") %>%
  row_spec(0, bold = TRUE, background = "#2C6E9E", color = "white") %>%
  footnote(general = "Green row = lowest xerror (best seed). Variable importance columns
   ↪  sum to ~1.",
           general_title = "Note: ", footnote_as_chunk = TRUE)
```

Code for Seed Sensitivity Table

```r
imp_vars <- c("TimeSearching_Min", "Years_Exp", "Salary", "Uses_AI", "Is_Remote")
```

```r
seed_plot <- seed_averages %>%
  select(seed, all_of(imp_vars)) %>%
  arrange(seed)

imp_mat <- as.matrix(seed_plot[, imp_vars])
imp_share <- imp_mat / rowSums(imp_mat)

matplot(
  seed_plot$seed,
  imp_share,
  type = "b",
  pch = c(19, 17, 15, 1, 2),
  lty = 1,
  lwd = c(2.6, 2.0, 2.0, 1.8, 1.8),
  col = c("#1E8449", "#2E86C1", "#7D3C98", "#C0392B", "#7F8C8D"),
  xlab = "Imputation Seed",
  ylab = "Relative Variable Importance (share)",
  main = "Variable Importance Stability Across Seeds"
)

legend(
  "topright",
  legend = c("TimeSearching_Min", "Years_Exp", "Salary", "Uses_AI", "Is_Remote"),
  col = c("#1E8449", "#2E86C1", "#7D3C98", "#C0392B", "#7F8C8D"),
  pch = c(19, 17, 15, 1, 2),
  lty = 1,
  lwd = c(2.6, 2.0, 2.0, 1.8, 1.8),
  bty = "n",
  cex = 0.8
)
```

Code for Seed Sensitivity Graph

```r
old_par <- par(mar = c(0, 1, 0.6, 1))
on.exit(par(old_par), add = TRUE)
plot.new()
plot.window(xlim = c(0, 10), ylim = c(0, 8))
box(col = "#666666")

draw_box <- function(xmin, ymin, xmax, ymax, label, fill, border, cex = 0.88) {
  rect(xmin, ymin, xmax, ymax, col = fill, border = border, lwd = 2)
  text(mean(c(xmin, xmax)), mean(c(ymin, ymax)), label, cex = cex, font = 2)
}
```

```r
draw_arrow <- function(x0, y0, x1, y1, label = NULL, lx = NULL, ly = NULL, col =
 ↪  "#666666", lty = 1) {
  arrows(x0, y0, x1, y1, length = 0.08, lwd = 1.8, col = col, lty = lty)
  if (!is.null(label)) {
    lx <- if (!is.null(lx)) lx else (x0 + x1) / 2
    ly <- if (!is.null(ly)) ly else (y0 + y1) / 2
    text(lx, ly, label, cex = 0.72, col = "#555555")
  }
}

draw_box(0.3, 3.2, 2.3, 4.8, "AI\nTool Adoption", "#DCEAF7", "#4A78A8")

draw_box(3.8, 5.2, 6.2, 6.8, "Financial\nCompensation", "#E8F4E3", "#5F8F4A")
draw_box(3.8, 1.8, 6.2, 3.2, "Workflow\nFriction",      "#F9E6D7", "#C77C3B")
draw_box(7.7, 3.2, 9.7, 4.8, "Job\nSatisfaction", "#EFE9FB", "#7A63A8")

draw_box(0.3, 0.2, 2.3, 1.2, "Remote\nStatus",       "#F1F1F1", "#808080", cex = 0.80)
draw_box(2.7, 0.2, 4.7, 1.2, "Years of\nExperience", "#F1F1F1", "#808080", cex = 0.80)


draw_arrow(2.3, 4.4, 3.8, 6.0, "May raise pay", lx = 2.9, ly = 5.5, col = "#5F8F4A")


draw_arrow(2.3, 3.6, 3.8, 2.8, "May increase friction", lx = 3.0, ly = 2.9, col =
 ↪  "#C77C3B")


draw_arrow(6.2, 6.0, 7.7, 4.6, "Higher pay may\nbuffer friction", lx = 7.2, ly = 5.8, col
 ↪  = "#5F8F4A")


draw_arrow(6.2, 2.5, 7.7, 3.5, "Friction may\nlower satisfaction", lx = 7.1, ly = 2.7,
 ↪  col = "#C77C3B")


lines(c(1.3, 1.3, 8.7, 8.7), c(0.2, 0.05, 0.05, 3.2), lty = 2, lwd = 1.5, col =
 ↪  "#888888")
arrows(8.7, 3.2, 8.7, 3.21, length = 0.08, lwd = 1.5, col = "#888888")


lines(c(3.7, 3.7, 9.0, 9.0), c(0.2, -0.05, -0.05, 3.2), lty = 2, lwd = 1.5, col =
 ↪  "#888888")
```

```
arrows(9.0, 3.2, 9.0, 3.21, length = 0.08, lwd = 1.5, col = "#888888")
```

Code for Conceptual Framework Diagram

Appendix B: Repository Information

All code, data, and analysis outputs are publicly available at:

https://github.com/peaceyyy/genai-productivity-satisfaction-tradeoff-study-data-analytics

Folder Structure The repository contains the following key directories:

- **`data/`** — Raw survey data and schema files
  - `survey_results_public.csv` — Stack Overflow 2024 Developer Survey (full)
  - `survey_results_schema.csv` — Column definitions and metadata
- **`imputation_scripts/`** — MICE imputation pipelines
  - `impute_data_cart.r` — CART imputation method
  - `impute_data_pmm.r` — Predictive Mean Matching method
  - `impute_data_rf.r` — Random Forest imputation method
- **`outputs/`** — Analysis results and model outputs
  - `data_imputed_primary.csv` — Final imputed dataset (seed=200)
  - `cart_cp_profile.csv` — Complexity parameter profiling results
  - `cart_hyperparam_shuffle.csv` — Hyperparameter tuning results
  - `cart_mice_seed_averages.csv` — Seed sensitivity summary statistics
  - `model_summaries/` — Model performance metrics by method
- **`Culminating Activity.Rmd`** — Main analysis document (this file)

All inline file references (e.g., `[outputs/cart_cp_profile.csv](...)`) point to files in this repository structure.

Appendix C: Notes on Random Seeds

Random seeds initialize pseudorandom number generators (PRNGs) so that stochastic processes (like MICE imputations or algorithmic subsampling) can be reproduced exactly. A seed is not a model parameter — it only fixes the RNG state for reproducibility. Because different seeds can produce slightly different imputations or model splits, reporting the seed and testing sensitivity across several seeds is good practice for transparency.

Why use `42`? The number 42 is a cultural reference (Douglas Adams' The Hitchhiker's Guide to the Galaxy) and has become a de-facto, tongue-in-cheek default in many code examples. There is no statistical or computational reason to prefer 42 over any other integer — it is purely conventional.